Reviewer Pack — Minimal Rank of Tropical Motives over Randomized Communicati…

Entry d2dba76a9240 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 08:20:47 UTC

Minimal Rank of Tropical Motives over Randomized Communication Complexity

Entry ID: `d2dba76a9240` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 08:20:47 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Motives) **Field B** (complexity object): Communication Complexity (Randomized Communication)

Statement:

> ['For a given explicit function f: {0,...,n-1} -> {0,1} in P, let M_f be the tropical motive associated with its truth-table. The randomized communication complexity of a function g is the minimum number of bits required for two parties to communicate an input x such that g(x) = 1 using randomness. Conjecture: For all functions f, the minimal rank of the tropical motive M_f is at least logarithmic in the randomized communication complexity of f, i.e., min_rank(M_f) ≥ log_2 CC_R(g).', 'Equivalently, for every function g with randomized communication complexity CC_R(g), there exists a tropical motive M_g such that its minimal rank satisfies min_rank(M_g) ≥ log_2 CC_R(g).', 'For any function f in P and any input size n ≤ 40, the association of f with its associated tropical motive M_f can be computed in polynomial time.']

Rationale (proposer's reasoning):

> ['Tropical motives provide a geometric interpretation of functions that could potentially reveal hidden structures in their complexity. By connecting this to randomized communication complexity, we aim to explore whether such geometric properties can help explain the lower bounds for communication complexity.', 'The logarithmic relationship suggests that the complexity of associating the tropical motive might be a bottleneck in determining the communication complexity of a function.', 'This conjecture aims to expose a new direction for understanding complexity by leveraging advanced algebraic geometry and providing a potential bridge between seemingly unrelated fields.']

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `652dfa99cd158914`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across 30 random seeds, at least 80% of generated functions f have a minimal rank of their associated tropical motive M_f that satisfies min_rank(M_f) ≥ log_2 CC_R(f). The conjecture is falsified if any seed produces a function f with min_rank(M_f) < log_2 CC_R(f).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "motives" AND "communication complexity" - "randomized communication complexity" AND "tropical motives" AND "minimal rank" - "polynomial time computation" AND "tropical motive" AND "communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=4.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = int(math.log2(len(f)))
        min_bits = float('inf')
        for i in range(2**n):
            x = [int(b) for b in format(i, f'0{n}b')]
            y = [int(b) for b in format(i + 1, f'0{n}b')]
            if f[i] == 1 and f[i + 1] == 0:
                min_bits = min(min_bits, math.ceil(math.log2(sum(abs(a - b) for a, b in zip(x, y))
        return min_bits

    def tropical_motive_rank(f):
        n = int(math.log2(len(f)))
        T = [[0 if i != j else 1 if f[i] == 1 else float('inf') for j in range(2**n)] for i in ran
        rank = 0
```

```
        for _ in range(2**n):
            min_val = float('inf')
            min_idx = -1
            for i in range(2**n):
                if T[i][i] == float('inf'):
                    continue
                for j in range(i + 1, 2**n):
                    if T[j][j] < min_val:
                        min_val = T[j][j]
                        min_idx = j
            if min_idx == -1:
                break
            rank += 1
            for i in range(2**n):
                T[i][min_idx] = float('inf')
                T[min_idx][i] = float('inf')
        return rank

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_random_function(n)
        cc = communication_complexity(f)
        mr = tropical_motive_rank(f)
        results.append({
            "n": n,
            "cc": cc,
            "mr": mr
        })

    min_mr = min(result["mr"] for result in results)
    max_cc = max(result["cc"] for result in results)

    conjecture_holds = min_mr >= math.log2(max_cc)
    counterexample = "" if conjecture_holds else f"min_rank(M_f)={min_mr}, log_2 CC_R(f)={math.log

    return {
        "metric_name": "minimal_rank",
        "metric_value": min_mr,
        "instances_tested": len(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
```

```
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db78fbb6.py", line 89, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db78fbb6.py", line 61, in run_trial
    cc = communication_complexity(f)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db78fbb6.py", line 30, in communication
    if f[i] == 1 and f[i + 1] == 0:
                     ~^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing any data, which means we cannot verify the conjecture's support or falsification based on the pre-registered criteria. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14636
2	propose	ollama_remote	glm4:latest	0	0	11685

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	preregistration	ollama_remote	glm4:latest	0	0	6267
4	novelty	ollama_remote	glm4:latest	0	0	4713
5	novelty	ollama_remote	glm4:latest	0	0	8515
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15670
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24467
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12275
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15586
10	judge	ollama_remote	glm4:latest	0	0	49770

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 163584 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d2dba76a9240.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d2dba76a9240.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d2dba76a9240.tar.gz` (if generated)